



UNIVERSIDAD DE CÓRDOBA

GRADO EN INGENIERÍA INFORMÁTICA
SISTEMAS EN TIEMPO REAL



UNIVERSIDAD DE CÓRDOBA

Autores:

Álvaro Palacios Criado p02pacra@uco.es

Profesor

José Manuel Palomares Muñoz

Córdoba, 23 de marzo de 2023

Índice

1. Programación con memoria compartida	2
1.1. Conceptos básicos de programas concurrentes	2
1.2. Paradigmas de programación concurrente	2
1.3. Ejecución atómica de instrucciones	3
1.4. Seguridad y Viveza	5
2. Secciones Críticas: Garantía de exclusión mutua	6
3. Cerrojos, Barreras y Semáforos	7
3.1. Cerrojos y Barreras	7
3.2. Semáforos	7
4. Monitores	8
5. Programación por Paso de Mensajes	8

Índice de figuras

1. Código de ejemplo de recursividad	2
2. Ejemplo de esquema de red P2P	3
3. Ejemplo de atomicidad	4
4. Ejemplo de conjuntos disjuntos	4
5. Ejemplo de afirmaciones debilitadas	5
6. Ejemplo de monitor para acceso al buffer	8
7. Diferencias entre memoria compartida y paso de mensajes	9
8. Diferencias entre los envíos	9

1. Programación con memoria compartida

1.1. Conceptos básicos de programas concurrentes

Concurrencia: dos o más procesos (concepto software) que se ejecutan simultáneamente trabajando de manera conjunta para realizar una tareas (concepto lógica). Una tarea tiene varios procesos → ejemplo de apagar el fuego.

Tipos de tareas:

- Independientes: los procesos asociados a las tareas no tiene contacto entre sí.
- Cooperativas: leer un dato, pasarlo a otro que lo interprete, y pasarlo a otra que decida según esa interpretación.
- Competitivo: Cuidado si dos quieren acceder al mismo recurso, hay conflicto (abrir y cerrar el agua).

Su comportamiento puede variar a lo largo del tiempo, suele ir de independientes a cooperativo/competitivo y vuelta a empezar. Una vez que tienen el dato pasan a ser independientes.

Lo ideal es que las tareas se ejecuten en paralelo, es decir, el orden de ejecución de las funciones individuales de cada tarea no importe. Siempre que sea posible y de manera ordenada cuando necesiten coordinarse (Todas las tareas paralelas son concurrentes). Comunicación y sincronización: variables compartidas y paso de mensajes.

1.2. Paradigmas de programación concurrente

Son los siguientes (va de grano fijo a grano grueso, que se esta paralelizando, cuanto mas grande sea el elemento a paralelizar, mas grande es el grano):

- **Paralelismo iterativo:** se hace cuando yo tengo una repetición conocida finita de una función, en la que se accede a diferentes datos según cada elemento (Ejemplo: hacer un negativo de la imagen). Cada for se realiza como una operación concurrente, hacemos "n" hilos, es decir, "n" elementos de la imagen. Simplifica la complejidad del bucle normal de $O(n)$, a $n \cdot O(1)$. (Optimización loop and rolling, complejidad de $O(n)$ pero sin salto por el incremento). De la manera que con el paralelismo se realiza en un ciclo. Es necesario que los datos sean accesibles individualmente, que no dependan unos de otros.
- **Paralelismo recursivo:** es a nivel de instrucción. Se trabaja sobre la función, por ejemplo, recursividad: tenemos un caso base al cual se llega a través de un árbol. La parte que no incluye recursividad se hace en paralelo (figura 1). Si hay mucha carga computacional en la función g del ejemplo 1 es muy eficiente. Para la dependencia de datos recursivos, se paraleliza la parte independiente.

```
int f(i){
    if(CasoBase){
        return CasoBaseValue;
    }
    else if(!CasoBase){
        return ((g(f(i-1)),i));
    }
    /*Es un bloque de codigo paralelo*/
    else {
        int a = f(i-1);    //Se crea hilo y no se detiene la ejecución
        int b = g(i);
        return a*b;        //Se hace stop esperando a "a"
    }
}
```

Figura 1: Código de ejemplo de recursividad

- **Productor/Consumidor:** es a nivel de función. Existen tareas que producen datos y otras que procesan esos datos de manera concurrente. La comunicación es en un único sentido. Se utiliza sobretodo cuando tenemos que conectar dos dispositivos a diferentes velocidades y hay un recurso accesible por un mismo elemento (R/W). Este paradigma cumple que siempre sea el resultado esperado. Para equiparar las diferentes velocidades se puede hacer lo siguiente: cuando se tenga un volumen de datos suficiente para procesar y solape con la generación del dato. (Ej: un chasis de coche tarda 3 meses y los coches se montan en dos días, se crea un almacén de chasis en común, y se van almacenando hasta poder montar tantos coches como para que la entrega de chasis solape con la construcción).
- **Cliente/Servidor:** es a nivel de función. Existen procesos de tipo servidor que atienden peticiones y clientes que piden. Es una comunicación bidireccional. Cliente pide (procesa datos) y el servidor ejecuta y devuelve resultados. Esto permite que el cliente siga ejecutando código o no, igual que el servidor. Cada cliente es un hilo nuevo (Sistemas distribuidos). Para no ser un cliente nuevo → cookies o inicio de sesión (Tabla hash). Si se accede a un dato común puede hacer un bloqueo.
- **Conjunto de compañeros (Peers):** es a nivel de proceso. Existen procesos independientes que se encargan de realizar una tarea de manera concurrente que se comunican intercambiando datos según los van necesitando. Redes P2P, desarrolla este paradigma. Cada nodo P2P puede ser, cliente si recibe información o servidor si sirve información (figura 2). SaaS, PaaS e IaaS son ejemplos de Peers. Bugs of task (Bolsa de tareas) → hay unas tareas y recursos. Reparte las tareas según los recursos (Ej: Cuadrilla de jardineros), el patrón dice que tareas tiene cada uno. Cada nodo coge el recurso, realiza la tarea (si necesita algo que no esta, espera, lo que llamamos decisiones dinámicas).



Figura 2: Ejemplo de esquema de red P2P

Paralelización de algoritmos: dos partes de un algoritmo son independientes si la intersección de sus conjuntos de escritura es el conjunto vacío y si la intersección entre el conjunto de escritura de cada uno de ellos con el conjunto de lectura del otro (en cada caso) es el conjunto vacío:

- Conjunto de lectura: todas aquellas variables que se lee el proceso son modificarlas a lo largo de la ejecución.
- Conjunto de escritura: todas aquellas variables que son creadas o modificadas por el proceso.

Cuando dos procesos no son totalmente independientes hay que arbitrar mecanismos para conseguir que la escritura y lectura de variables compartidas no se acceda simultáneamente.

1.3. Ejecución atómica de instrucciones

Se denomina **ejecución atómica** como la ejecución de una acción única indivisible. En ensamblador aunque una instrucción puede durar varios ciclos, se garantiza que no se interrumpirá, es decir, la ejecución atómica. Pero en alto nivel no es así, ya que una simple instrucción de inicializar una variable ($a=3$) puede ser interrumpida ya que en ensamblador requiere de dos instrucciones. Tipos de atomicidad:

- Paralelismo de Grano Fino: la ejecución atómica de las instrucciones está garantizada mediante mecanismo hardware. Se consigue en ensamblador.
- Paralelismo de Grano Grueso: se garantiza mediante mecanismos software:
 - Grano Medio: hace referencia a bloques de código paralelo dentro de una misma tarea. Alto nivel de cooperación/competición → mucha sincronización.
 - Grano Grueso: hace referencia a la ejecución en paralelo de diversas tareas dentro de un mismo sistema. Bajo nivel de cooperación/competición → poca sincronización.

Vamos a analizar un ejemplo (figura 3): al ejecutarse de manera automática todo el cuerpo del bucle es equivalente a ejecutarla secuencialmente. Si no fuera de esta manera, es decir, fuera concurrente el orden sería aleatorio. En este caso es realizar una doble evaluación de la sentencia condicional *if* (junto encima de EJECUCIÓN ATÓMICA). La primera evaluación se puede realizar en paralelo de esta manera conseguirá eliminar una gran parte de las tareas. La segunda evaluación tiene que realizarse junto con la asignación de manera atómica para garantizar que entre medias no interrumpe nada.

```

#define TAM n
int m = 0; // Compartida Escritura
int a[TAM]; // Compartida Lectura
int i; // Privada

omp_set_num_threads(TAM);
#pragma omp parallel shared(m,a)
private(i)
{
    #pragma omp for schedule(dynamic)
    nowait
    {
        for (i=0; i < TAM; i++){
            EJECUCION ATOMICA {
                if (a[i] > m)
                    m = a[i];
            }
        }
    }
}

MOV SI, 0 ; (i = 0)
BUCLE_FOR: CLI ; BLOQUEO INTERRUPCIONES
MOV AX, A[SI] ; a[i]
CMP AX, [m] ; (a[i] > m)
JBE CONTINUA
MOV [m], AX ; m = a[i]
CONTINUA: STI ; HABILITAR INTERRUP.
INC SI ; i++
; i < TAM => Bucle for
CMP SI, TAM
JB BUCLE_FOR

```

Figura 3: Ejemplo de atomicidad

Interferencia entre tareas: un conjunto de tareas se consideran libres de interferencias si ninguna asignación en ninguna tarea impide el cumplimiento de las afirmaciones críticas (postcondiciones) de otras tareas. Cuanto más independientes sean las tareas más eficaz es, todas las combinaciones posibles entre el grano fino de las tareas siempre da el mismo resultado, cuanto más simples mejor. Hay técnicas para evitar la interferencia entre tareas:

- **Variables disjuntas:** los conjuntos de escritura de las áreas no se solapan. Tener que ejecutar todas las tareas para saber que vale las variables. Ejemplo del Algoritmo de Sovel que realiza cambios de la imagen en su contorno, al necesitar 3 píxeles adyacentes, tenemos cooperación. Aplicamos mecanismo fork-join (división-unión). El ultimo for no se ejecuta hasta tener las 3 secciones, la postcondición se hace a nivel de bloque. Consume mucha memoria (es un EMP) (figura 4):

● **Calcular:** $B[i] = A[i-1]*-1 + A[i]*2 + A[i+1]*-1;$

```

int A[TAM], b1[TAM], b2[TAM], b3[TAM];
int B[TAM];
int i;

for (i=0; i < TAM/2; i++)
    b1[i]=b2[i]=0;

#pragma omp parallel shared(A,b1,b2,b3)
private(i)
{
    #pragma omp sections
    {
        #pragma omp section
        {
            for (i=0; i<TAM-1; i++)
                b1[i+1] = A[i]*(-1);
        }

        #pragma omp section
        {
            for (i=1; i<TAM; i++)
                b3[i-1] = A[i]*(-1);
        }
    }

    for (i=0; i<TAM; i++)
        B[i] = b1[i]+b2[i]+b3[i];
}

```

Figura 4: Ejemplo de conjuntos disjuntos

- **Afirmaciones debilitadas:** cuando hay solapamientos entre los conjuntos de escritura y lectura de las tareas, se pueden asumir afirmaciones (postcondiciones) menos estrictas que tengan en cuenta

la ejecución concurrente. Ejemplo (figura ??) → podemos asegurar que después de la ejecución de las 2 tareas $x=3$, si ponemos interrupciones entre los incrementos no podemos afirmar parcialmente el valor de x .

```
int x = 0;

#pragma omp parallel sections shared(x)
{
    #pragma omp section
    { // Al inicio x = 0 || x = 2
        #pragma omp atomic
        x++;
    } // Al final x = 1 || x = 3

    #pragma omp section
    { // Al inicio x = 0 || x = 1
        #pragma omp atomic
        x += 2;
    } // Al final x = 2 || x = 3
}

// Como afirmación debilitada podemos afirmar que x = 3;
```

Figura 5: Ejemplo de afirmaciones debilitadas

- **Invariantes globales:** emplear invariantes globales que capturen las relaciones entre las variables globales en todo momento. Garantizan las postcondiciones. Se construye el código para cumplir esas invariantes.
- **Sincronización:** utilizar mecanismos que ordenen la ejecución de los bloques de código para impedir su ejecución concurrente. **EMP** → Sistema Embarazosamente Masivo Paralelo, no importa la técnica ya que es 99,999% independiente y solo confluye en un punto en el que se compite o se coopera. Podemos encontrar mucho tipos de sentencias que consiguen que bloques de sentencias de una tarea aparezcan como unidades indivisibles. Estas acciones atómicas de grano grueso llevan a usar la resincronización de dos maneras:
 - Exclusión mutua: impedir que dos o más tareas ejecuten simultáneamente un determinado bloque de código con sentencias críticas.
 - Sincronización de condiciones: impedir que una tarea comience a ejecutar un bloque de código antes de que se cumpla una determinada condición (sincronizar etapas).

1.4. Seguridad y Viveza

Son propiedades que deben ser atributos ciertos en cada posible estado de una tarea.

- **Seguridad:** determina que en ningún momento la ejecución es interrumpida anormalmente y que es capaz de finalizar proporcionando un resultado correcto. (Nada malo pasa en la ejecución). Tenemos:
 - Garantía de exclusión mutua: debe garantizar que un trozo de código crítico va a ser ejecutado por una única tarea concurrente cada vez.
 - Ausencia de deadlock (interbloqueos): debe garantizar que o todas las tareas van a estar esperando infinitamente a que se produzca una determinada acción.
 - Terminación normal: debe garantizar que el programa finaliza o vuelve a un estado de espera análogo al inicial, proporcionando una respuesta correcta en todo momento.
 - Reanudación análoga: debe garantizar que al finalizar su ejecución, el programa puede volver a ser invocado manteniendo un comportamiento análogo en función del estado externo.

- **Viveza:** determina que la tarea debe ser capaz de progresar en algún momento de la ejecución. que algo bueno ocurre en algún momento de la ejecución. Tenemos:
 - Política de planificación con equidad incondicional: cuando todas las acciones atómicas elegibles son ejecutadas al menos eventualmente.
 - Política de planificación con equidad débil: en el caso de acciones atómicas condicionales (dependientes de una determinada condición), si cumple la equidad incondicional y se ejecuta cuando su condición pasa a ser verdadera y se mantiene como verdadera al menos hasta que es chequeada por el proceso que ejecuta la acción atómica condicional.
 - Política de planificación con equidad fuerte: si tiene equidad incondicional y si la condición pasa a ser cierta un número infinito de veces, aunque cambie el valor de su condición.

2. Secciones Críticas: Garantía de exclusión mutua

Garantía de exclusión mutua: problemas de la modificación de variables compartidas → condiciones de carrera. Cuando dos tareas concurrentes tratan de acceder a una variable compartida, compiten en el acceso según su velocidad de ejecución → condición de carrera.

Reglas necesarias de las secciones críticas:

- **Exclusión mutua:** solo una tarea debe encontrarse dentro de la sección crítica.
- **Ausencia de Deadlock/Livelock:** si dos o más tareas están intentando entrar en su sección crítica libre, una de ellas conseguirá el acceso. Además, ninguna tarea podrá bloquearse dentro de su sección crítica.
- **Ausencia de retardos innecesarios:** si una tarea solicita acceder a la sección crítica libre, dicha tarea accederá en el menor número de ciclos posible.
- **Entrada eventual:** todos los procesos que deseen entrar en su sección crítica conseguirán acceso a la misma en algún instante de su ejecución.

Implementaciones:

- **Busy waiting:** Dentro de este tipo se agrupan todas aquellas implementaciones en las que las tareas que están esperando para entrar a la sección crítica están consumiendo ciclos.
 - **Inhabilitación de interrupciones:** Mediante este mecanismo se impide la conmutación de tareas. Es una solución excesivamente drástica ya que impide que ninguna otra tarea pueda actuar con las interrupciones, pudiéndose quedar bloqueada. No es útil en sistemas multiprocesador, ya que solo impide el acceso a las tareas en ejecución en el mismo procesador. Se utiliza exclusivamente dentro de las tareas que componen el Sistema Operativo.
 - **Alternancia estricta:** Es una solución de grano grueso que emplea 1 variable. No es buena solución porque aunque garantiza exclusión mutua, el resto de reglas no se cumplen: una tarea muy lenta podría bloquear a otra muy rápida fuera de sección crítica. Ejemplo de usar turnos de Sistemas Operativos.
 - **Cerrojos:** Es una solución de grano grueso que emplea 2 variables (para generalizarlo a n procesos, serán necesarias n variables). El hardware debe implementar una instrucción atómica que sea capaz de comprobar el estado de una variable y realizar la modificación de la misma.
 - **Instrucciones Test and Set Lock (TSL):** Es una solución de grano fino. Implementa en hardware de manera atómica la lectura de una posición de memoria, colocarlo en un registro y almacenar un valor distinto de cero en la posición de memoria original. Guarda en memoria de manera atómica un valor a true. Comportamiento normal, alguna burbuja pero cada 3 instrucciones no en cada una.
 - **Instrucciones Fetch and Add (FA):** Es una solución de grano fino. Implementa en hardware de manera atómica la lectura de una posición de memoria, incrementar el contenido de dicha posición y almacenar dicha posición de memoria original, devolviendo el valor antes de incrementar en una variable local. Desventaja: como siempre hace incremento, si no entra tenemos que hacer el decremento.

- **Dormir/Despertar:** Este tipo de implementación incluye todas aquellas aproximaciones en las que las tareas en espera pasan a condición de no ejecutables y no consumen ciclos. El S.O. es el encargado de despertarlas y darles acceso a la sección crítica en cuanto ésta se quede libre. Las tareas que no tengan que ver con la sección crítica pueden interrumpir, y las que sí solo si se planifican. Si una quiere entrar pero no puede, pide ser bloqueada. Garantiza entrada eventual, aunque tiene un retardo algo mayor, exclusión mutua.
 - **Primitivas Sleep/Wakeup:** Es una solución de grano grueso. Sleep: Llamada al sistema que provoca la suspensión de la tarea que lo invoca hasta que otro lo despierte. Wakeup: Llamada al sistema que despierta al proceso que se indica. Pueden existir problemas de pérdidas de llamadas wakeup en los casos en los que las tareas cambien de contexto entre que se chequee una variable y se decida que se ha de realizar una llamada sleep. Puede provocar interbloqueos, para ello no haya que separar la evaluación de la ejecución. Evaluación + Sleep/Wakeup de manera atómica → garantía de exclusión mutua.

3. Cerrojos, Barreras y Semáforos

3.1. Cerrojos y Barreras

Cerrojos: Los cerrojos se han mostrado con anterioridad y es el mecanismo más simple para controlar el acceso a las secciones críticas.

- No tienen control de tareas bloqueadas esperando a que se libere el cerrojo → No permiten políticas de prioridad de tareas.
- No permiten que puedan entrar más de una tarea a una sección protegida de código, aunque ésta lo admita → Dificultad para sincronizar más de 2 tareas a la vez.

Las barreras permiten sincronizar muchas tareas a la vez: Bloquean todas las tareas hasta que se encuentren esperando n tareas. Permiten sincronizar diferentes etapas de un algoritmo.

Barreras: tenemos dos técnicas:

- **Contadores compartidos:** Pueden presentar problemas de reanudación análoga tras su uso (hay que realizar una puesta a cero, en sección crítica). Para evitarlo se puede asumir que el último que sea capaz de “pasar sin espera”, realice de manera atómica una puesta a cero → Complejidad y retardo innecesario.
- **Mediante banderas y coordinadores:** En lugar de incrementar un contador, cada vez que una tarea llegue a una barrera, activa una bandera en un vector de banderas compartidas. Existe una tarea especial llamada “coordinador” que se encarga de ir chequeando sistemáticamente dicho vector de banderas y mientras que haya alguna bandera inactiva, no permite continuar al resto de tareas. Este método permite el control de prioridad e incluso es capaz de reanudar en caso de bloqueo prolongado de alguna tarea. (Fork-Join)

3.2. Semáforos

Semáforos Binarios: Los semáforos binarios están compuestos por una variable booleana y dos instrucciones atómicas:

- UP(semáforo): pone el valor del semáforo a uno.
- DOWN(semáforo): pone el valor del semáforo a cero. En caso de que el semáforo ya tuviese el valor 0, la tarea se bloquea hasta que en algún momento el semáforo tome valor a uno.

La reactivación de las tareas bloqueadas depende del sistema, que será el encargado de escoger una de las tareas bloqueadas y reactivarla.

Semáforos multievaluados: Una generalización de los semáforos binarios son los semáforos multivaluados. Estos utilizan una variable entera no negativa y dos instrucciones atómicas:

- UP(semáforo): Incrementa en uno el valor del semáforo.
- DOWN(semáforo): Decrementa en uno el valor del semáforo. Si antes de incrementar el valor del semáforo es cero, la tarea se bloquea.

Mediante esta generalización se puede permitir el paso a más de una tarea. Mediante la inicialización del semáforo a un valor entero superior a cero se controla cuántas tareas pueden solicitar la entrada. Suelen estar implementados como llamadas al sistema con inhibición de interrupciones y/o utilizando la instrucción en ensamblador TSL (para los semáforos binarios) o la instrucción FA (para los semáforos multivaluados), además de utilizar el mecanismo sleep/wakeup de manera protegida dentro del Sistema Operativo.

4. Monitores

Los monitores (figure 6) son un conjunto de procedimientos, variables y estructuras de datos agrupados en forma de clase o módulo especial dentro de los lenguajes orientados a objetos. Todas las variables están definidas como privadas y no son accesibles desde fuera de los monitores. Los monitores solo permiten que una tarea esté ejecutándose dentro de ella. Cuando una tarea intenta acceder a un monitor, lo primero que hace es comprobar si hay alguna otra tarea dentro, en cuyo caso se suspende hasta que el otro proceso salga. Suelen tener dos funciones miembros que controlan las variables de condición de los monitores:

- WAIT(condición): La tarea se bloquea en la lista de espera asociada a la variable condición. En ese momento libera el monitor para que otra tarea pueda entrar al monitor.
- SIGNAL(condición): El sistema despierta a una de las tareas que se encuentre bloqueada en la lista de la variable condición.

Conceptualmente son similares a SLEEP/WAKEUP.

```
monitor Buffer {

    char buf[n];
    int  front = 0, rear = 0, count = 0;
    cond not_full, not_empty;

    void putElement (char c) {
        while (count == n) wait (not_full);
        buf[rear] = c; rear = (rear + 1) % n; count++;
        signal (not_empty);
    }
    char getElement (void) {
        char element;
        while (count == 0) wait (not_empty);
        element = buf[front]; front = (front + 1) % n; count--;
        signal (not_full);
        return (element);
    }
}
```

Figura 6: Ejemplo de monitor para acceso al buffer

5. Programación por Paso de Mensajes

Para sistemas Multicomputador (Sistemas Distribuidos) → un computador compuesto de varias CPUs. Es necesario un emisor, receptor, protocolo, acceso compartido al medio y un mensaje. Diferencias entre memoria compartida y paso de mensajes (figura 7):

Multiprocesador Mem compartida	Paso de Mensajes
Difícil de entender	Fácil de entender
Más rápido	Menos rápido
Grano fino	Grano grueso

Figura 7: Diferencias entre memoria compartida y paso de mensajes

Conceptos básicos:

- En la programación distribuida las tareas no tienen acceso a variables compartidas: cada tarea tiene su propia memoria local totalmente independiente del resto.
- La sincronización se realiza mediante comunicación explícita entre tareas: unas tareas envían datos a otras tareas, que deben estar esperando dichos envíos.
- Las recepciones pueden ser identificadas, provenientes de otra tarea específica, o bien generales, provenientes de cualquier otra tarea.
- Es la forma más natural de programación ya que imita el comportamiento humano de paso de turno.

La sincronización se realiza debido a que las instrucciones de envío/recepción son bloqueantes. Hay tres tipos de envíos (figura 8):

- **Asíncronas o No Bloqueantes:** El emisor continúa el proceso independientemente de si el receptor ha recibido o no el mensaje. (Estándar POSIX)
- **Síncronas o Bloqueantes:** El emisor continúa procesando solo cuando el mensaje ha sido recibido por el receptor.
- **Invocación remota:** El emisor continúa procesando cuando recibe un mensaje de confirmación de recepción por parte del receptor (Implementado en ADA)

Todas las recepciones son síncronas ya que una tarea no puede procesar un mensaje que no ha sido recibido. La tarea que espera recibir se bloquea hasta que recibe un cierto dato. Utilizando las recepciones síncronas generales se puede controlar el acceso a las secciones críticas de código o implementar barreras de sincronización.

Síncrono(Bloqueante)	Asíncrono(No bloqueante)	Invocación Remota
Emisor cierra cuando sabe que a recibido el mensaje (llamada de telefono). No se sabe si el receptor lo ha entendido.	Emisor envía y no se garantiza la llegada del mensaje (correo convencional). Se puede mejorar con un certificado de que ha entregado. Estándar POSIX: envía y sigue	Emisor envía y no continúa hasta que el receptor confirma que recibe y entiende el mensaje.

Figura 8: Diferencias entre los envíos

Los SMS lo envía a cada torre de telefonía para saber quien esta en cada celda y aprovecha el pipeline para enviar datos.

Técnicas:

- **Llamadas a procedimientos remotos (RPC):** Es un mecanismo de ejecución de procedimientos basado en el protocolo cliente-servidor. En el mecanismo cliente-servidor: el servidor exporta una serie de funciones y los clientes solicitan la ejecución de dicha función, bloqueando mientras tanto la ejecución del código de la tarea cliente. Cuando se realiza una llamada RPC a un servidor, se crea un hilo que atiende dicha petición, ejecuta la función asociada, envía la respuesta al cliente y finaliza dicho hilo, liberando todos los recursos utilizados. Cada petición es nueva, con lo que se inventan las cookies y login, ya que HTTP es stateless y no es eficiente.

- **Rendezvous:** El mecanismo RPC proporciona comunicación y exclusión mutua, pero no proporciona de manera directa sincronización. El mecanismo de Rendezvous (en francés, cita o punto de encuentro) permite además de la comunicación entre tareas, la sincronización entre las mismas (devuelve información $\rightarrow$ invocación remota). Tras la ejecución de un Rendezvous la tarea llamadora y la tarea receptora continúan su ejecución a la vez de manera síncrona (fork-join).